

Procesarea evenimentelor. Șablonul MVC. FXML

Conf. Univ. Arthur Molnar

Universitatea Babeș-Bolyai

2025

Privire de ansamblu

- 1 Procesarea evenimentelor
 - Livrarea evenimentelor (eng. event delivery)
 - Tratarea evenimentelor
 - Metode de conveniență
- 2 Model-View-Controller
- 3 FXML

Evenimente I

- Aceste slide-uri sunt bazate pe documentația disponibilă la [JavaFX: Tratarea Evenimentelor](#).
- Fiecare acțiune a utilizatorului generează un eveniment în interfața grafică, ex.:
 - apăsarea sau eliberarea unei taste;
 - mișcarea mouse-ului, apăsarea sau eliberarea unui buton;
 - deschiderea sau închiderea unei ferestre;
 - derularea ferestrei (eng. *scroll*);
- Evenimentele din JavaFX sunt clase derivate din clasa [javafx.event.Event](#).
- Fiecare eveniment are:
 - *tip*: o instanță a clasei [EventType](#), ex.:
 - [KeyEvent.KEY_PRESSED](#)
 - [MouseEvent.MOUSE_RELEASED](#)
 - [WindowEvent.WINDOW_SHOWN](#)

Evenimente II

- *sursă*: originea evenimentului, considerând locația evenimentului în lanțul de tratare al evenimentelor (eng. *dispatch chain*); sursa evenimentului este actualizată în timpul parcurgerii lanțului de tratare.
- *țintă*: nodul pe care a avut loc acțiunea; acesta nu poate fi modificat.
- Clasele care implementează interfața `EventTarget` pot fi ținta unui eveniment, ca de exemplu `Window`, `Scene` sau `Node`.

Procesul de livrare al evenimentelor (eng. *event delivery*)

- După generarea unui eveniment, au loc mai multe faze în tratarea acestuia:
 - 1 Selectarea țintei (eng. *target selection*)
 - 2 Construirea rutei (eng. *route construction*)
 - 3 Captarea evenimentelor (eng. *event capturing*)
 - 4 Transmiterea către nodul rădăcină (eng. *event bubbling*)

Selectarea țintei (eng. *target selection*)

- Ținta evenimentului e determinată în concordanță cu un set de reguli, de exemplu:
 - Pentru evenimentele generate de taste - ținta este nodul care are focusul aplicației.
 - Pentru evenimentele generate de mouse - ținta este nodul de la locația cursorului.
 - Pentru gesturile continue, ca de exemplu cele generate pe un ecran sensibil la atingere - ținta este nodul de la punctul central al tuturor punctelor atinse în momentul demarării gestului.
- Dacă la locația țintei se află mai mult de un nod, cel mai de deasupra (eng. *topmost*) este considerat ținta.

Construirea rutei (eng. *route construction*)



Figure: Sursa figurii: [Exemplu de interfață grafică.](#)

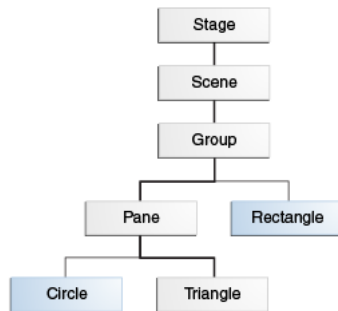


Figure: Sursa figurii: [Lanț de tratare al evenimentelor.](#)

- Ruta implicită a unui eveniment este calea de la obiectul [Stage](#) către nodul dat.

Captarea evenimentelor (eng. *event capturing*)

- În această fază, evenimentul e transmis în jos pe ierarhia de componente către nodul țintă.
- Dacă pe această cale există filtre care îl procesează, filtrele sunt apelate după care continuă transmiterea evenimentului.
- Dacă unul din filtre consumă evenimentul, acesta nu mai este transmis nodului următor.
- Altfel, evenimentul este transmis mai departe până își atinge ținta, unde este procesat.

Transmiterea către nodul rădăcină (eng. *event bubbling*)

- În această fază evenimentul e transmis în sens opus, de la țintă înapoi către nodul rădăcină.
- Dacă ținta sau orice alt nod de pe ierarhia de transmitere are o metodă de tratare a evenimentului (eng. *event handler*) atunci acesta este apelat, după care evenimentul e transmis nodului următor.
- Dacă una din funcțiile de tratare consumă evenimentul, transmiterea acestuia se încheie.
- Dacă nici una din funcțiile de tratare nu a consumat evenimentul, acesta ajunge la nodul rădăcină, iar tratarea sa se încheie.

Tratarea evenimentelor I

- Aceasta se face prin utilizarea de filtre (eng. *filters*) și funcții de tratare (eng. *event handlers*), ambele implementând interfața [EventHandler](#).
- Filtrele și/sau funcțiile de tratare trebuie să fie înregistrate pentru un anumit tip de eveniment pentru a-l putea procesa.
- Filtrele sunt executate în faza de captare a evenimentelor, iar funcțiile de tratare în momentul transmiterii către nodul rădăcină.
- Astfel, diferența dintre filtre și funcții de tratare e dată de momentul execuției acestora.
- Un filtru al unui nod părinte poate procesa evenimentele generate de mai mulți copii ai componentei.

Tratarea evenimentelor II

- Un nod poate adăuga mai multe filtre.
- În momentul în care un eveniment trece printr-un nod care a înregistrat un filtru pentru acesta, filtrul este executat. Evenimentul poate fi astfel consumat de filtru sau poate fi transmis către următorul nod inferior în ierarhie.
- În momentul în care un eveniment trece printr-un nod care a înregistrat o metodă de tratare a acestuia (eng. *handler*), această metodă este executată iar evenimentul trece către următorul nod superior în ierarhie.
- Un nod poate înregistra mai multe metode pentru tratarea unui eveniment.

Tratarea evenimentelor III

- În cazul în care un eveniment a fost consumat de către un filtru sau o metodă de tratare, acesta nu va fi transmis mai departe către nodurile inferioare/superioare din ierarhie.
- Consumarea unui eveniment de către un filtru implică faptul că nici unul din nodurile copil nu vor putea îndeplini acțiuni pe baza evenimentului. Consumarea unui eveniment de către o metodă de tratare implică faptul că nici unul din nodurile părinte nu vor putea îndeplini acțiuni pe baza evenimentului.
- Un eveniment poate fi consumat prin apelarea metodei `consume()`.
- Metodele de tratare a evenimentului definiți implicit pentru controalele UI JavaFX consumă în mod tipic cele mai multe evenimente de intrare.

Filtrele de evenimente (eng. *event filters*)

- Un filtru poate fi utilizat pentru mai multe noduri și tipuri de eveniment.
- Nodurile părinte pot utiliza filtrele de evenimente pentru a furniza nodurilor copil un mecanism comun de procesare al evenimentelor.
- Pentru a procesa un eveniment în timpul fazei de captură a evenimentelor, nodul interesat trebuie să înregistreze un filtru de evenimente.
- Codul executat în momentul interceptării evenimentului trebuie adăugat în implementarea metodei `handle()`. Aceasta este definită în cadrul interfeței `javafx.event.EventHandler`.
- Metoda `addEventFilter()` trebuie utilizată pentru a înregistra un filtru pentru evenimente.
- Metoda `removeEventFilter()` trebuie utilizată pentru a înlătura un filtru pentru evenimente; acest lucru va împiedica pe viitor procesarea evenimentului.

Metode de tratare a evenimentelor (eng. *event handlers*)

- O metodă de tratare (eng. *handler*) poate fi utilizată pentru mai multe noduri și tipuri de eveniment.
- Nodurile părinte pot utiliza aceste metode pentru a furniza nodurilor copil un mecanism comun de procesare al evenimentelor.
- Pentru a procesa un eveniment în timpul transmiterii către nodul rădăcină (eng. *event bubbling*), nodul interesat trebuie să înregistreze o metodă de tratare a evenimentului.
- Codul executat în momentul interceptării evenimentului trebuie adăugat în implementarea metodei `handle()`. Aceasta este definită în cadrul interfeței `javafx.event.EventHandler`.
- Metoda `addEventFilter()` trebuie utilizată pentru a înregistra o metodă de tratare a evenimentului.
- Metoda `removeEventFilter()` trebuie utilizată pentru a înlătura o metodă de tratare a evenimentului; acest lucru va împiedica pe viitor procesarea evenimentului.

Metode de conveniență I

- Unele clase JavaFX definesc metode de tratare a evenimentelor ale căror proprietăți pot fi atribuite folosind metode de tip *setter*.
- Aceste metode se numesc metode de conveniență (eng. *convenience methods*).
- Acest mecanism oferă un mod facil de a înregistra ușor metodele de tratare a evenimentelor.
- Clase precum **Node**, **Scene**, **Window** și subclasele lor se regăsesc printre cele care definesc acest tip de metode.
- O metodă de conveniență are următorul format:

```
setOn<Event-type>(  
    EventHandler<? super event-class> value)
```

Metode de conveniență II

- unde:
 - *Event-type* este tipul evenimentului, ex. pentru evenimente de tipul `MOUSE_CLICKED`: `setOnMouseClicked`.
 - Metoda acceptă o clasă ce conține metoda care tratează evenimentul pentru o clasă ce reprezintă un eveniment (ex. `KeyEvent`) sau pentru oricare din clasele sale părinte.
- De exemplu:

```
setOnKeyTyped(  
    EventHandler<? super KeyEvent> value)
```


Model-View-Controller (MVC) I

Este un șablon de proiectare arhitectural ce are ca scop separarea responsabilităților din cadrul aplicației.

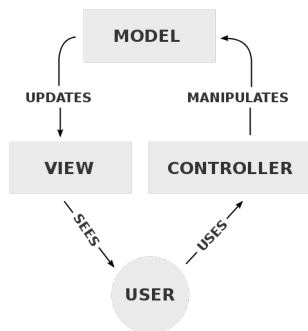


Figure: Sursa figurii: [Wikipedia](#)

Model-View-Controller (MVC) II

Modelul

- Reprezintă și gestionează datele ce țin de domeniul aplicației.
- Este responsabil pentru:
 - Încărcarea datelor necesare pentru componenta de tip *View*
 - Salvarea modificărilor cerute de componenta de tip *Controller*

Model-View-Controller (MVC) III

View

- Prezintă datele din domeniu utilizatorului.
- În cazul în care setul de date este mare, doar un subset al acestora este vizibil. În acest caz, componenta *View* va cere încărcarea doar acelui subset de date care este vizibil (ex. o componentă de tipul "*infinite scroll*" va încărca tot timpul doar datele care urmează să fie vizibile utilizatorului)

Controller

- Mediază între utilizator și componenta având rolul de *View*.
- Interpretează input-ul utilizatorului și emite instrucțiuni către componentele *Model* sau *View* pentru ca acestea să se actualizeze în mod corespunzător.
- Convertește acțiunile utilizatorului (care vin din componenta *View*) în cereri de navigare sau editare a datelor.

FXML

- FXML este un limbaj de adnotare declarativ, bazat pe limbajul XML.
- Este utilizat pentru a proiecta interfețe grafice cu utilizatorul (eng. graphical user interfaces, GUI) fără ca aplicația să necesite recompilare odată cu fiecare modificare a elementelor acesteia.
- În acest mod se realizează separarea dintre *nivelul de prezentare* și *nivelul logic* al aplicației.
- *Scene Builder* este o unealtă vizuală pentru dispunerea elementelor interfeței grafice care permite proiectarea și implementarea rapidă a acestora, fără a fi nevoie de scrierea de cod sursă; [unealta Scene Builder](#), sau [plugin-ul Scene Builder pentru IntelliJ](#).
- Utilizatorii pot adăuga componente folosind drag & drop, iar proprietățile acestora pot fi modificate folosind un mediu vizual, după care codul FXML este generat în mod automat de către unealtă.
- Codul FXML astfel generat poate fi apoi combinat cu un proiect Java prin adăugarea la interfața grafică a logicii programului.

Programatic și declarativ I



Figure: Sursa figurii: Notițe de curs MAP (Camelia Serban)

Programatic și declarativ II

Programatic

```
BorderPane border = new BorderPane();  
Label top = new Label("Page Title");  
border.setTop(top);  
Label center = new Label("Some data here");  
border.setCenter(center);
```

Programatic și declarativ III

Declarativ

```
<BorderPane>  
  <top>  
    <Label text="Page Title"/>  
  </top>  
  <center>  
    <Label text="Some data here"/>  
  </center>  
</BorderPane>
```

Elementele FXML I

- Următoarele informații sunt bazate pe documentația disponibilă la: [Utilizarea FXML pentru crearea unei interfețe grafice](#).
- Pentru a crea interfața următoare, se utilizează Scene Builder. Exemple de fișiere FXML sunt disponibile în cadrul materialelor de curs.

Elementele FXML II

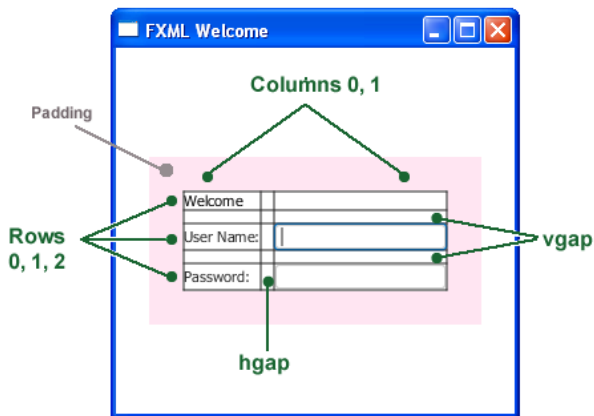


Figure: Sursa figurii: Formular de login

CSS

- Pentru a aplica un stil se folosește un fișier de tipul *css*.
- Fișierul trebuie să fie creat și salvat în cadrul proiectului.

Clasa javafx.fxml.FXMLLoader

```
Parent root = FXMLLoader.load(getClass().getResource(  
                                "sample.fxml"));  
stage.setScene(new Scene(root, 550, 350));
```

or

```
FXMLLoader loader = new FXMLLoader();  
loader.setLocation(getClass().getResource("sample.fxml"));  
GridPane root = (GridPane) loader.load();  
stage.setScene(new Scene(root, 550, 350));
```

Controller-ul din cadrul FXML I

```
<GridPane hgap="10" maxHeight="-Infinity" maxWidth="-Infinity"  
minHeight="-Infinity" minWidth="-Infinity" prefHeight="400.0"  
prefWidth="600.0" stylesheets="@Login.css" vgap="10.0"  
xmlns:fx="http://javafx.com/fxml/1"  
xmlns="http://javafx.com/javafx/11.0.1"  
fx:controller="sample.Controller">
```

Controller-ul din cadrul FXML II

- Numele clasei controller trebuie specificat în cadrul fișierului `.fxml`.
- În exemplul curent, această clasă se numește `sample.Controller`.
- Clasa are rolul de a gestiona tratarea evenimentelor din cadrul interfeței grafice.
- Controalele interfeței grafice pot fi accesate și utilizate în cadrul clasei cu rol de controller prin intermediul câmpului `fx:id`.

```
<TextField fx:id="userInput" promptText="username"  
GridPane.columnIndex="1" GridPane.rowIndex="1">
```

Controller-ul din cadrul FXML III

```
public class Controller {  
    @FXML private TextField userInput;  
    @FXML private PasswordField passwordInput;  
    @FXML private Button signInButton;  
  
    // ...  
}
```

Controller-ul din cadrul FXML IV

Gestionarea evenimentelor în cadrul clasei Controller

- Pentru fiecare element al interfeței grafice putem specifica care metodă îi va trata evenimentele, pentru fiecare eveniment ce poate avea loc.
- Aceste metode trebuie să fie implementate în cadrul clasei Controller.

```
<Button fx:id="signInButton"  
onAction="#handleSignInButtonClick" text="Sign in"  
textAlignment="CENTER" GridPane.columnIndex="1"  
GridPane.rowIndex="3">
```

Controller-ul din cadrul FXML V

```
public class Controller {  
    private static String USER_NAME = "map_user";  
    private static String PASSWORD = "map_pass";  
  
    @FXML private TextField userInput;  
    @FXML private PasswordField passwordInput;  
    @FXML private Button signInButton;  
  
    @FXML  
    private void handleSignInButtonClick(ActionEvent e)  
    {  
        if (this.userInput.getText().equals(USER_NAME) &&  
            this.passwordInput.getText().equals(PASSWORD))  
        {  
            Alert alert = new Alert(  
                Alert.AlertType.INFORMATION);  
            alert.setTitle("All is well");  
            alert.setHeaderText(  

```


Controller-ul din cadrul FXML VI

```
        "Great! User and password validated.");  
        alert.showAndWait();  
    }  
    else  
    {  
        // ERROR!  
    }  
}
```

Rezumat I

- Acțiunile utilizatorului aplicației generează evenimente, dintre care unele trebuie gestionate de către program.
- În cadrul de aplicație JavaFX acest lucru este implementat prin utilizarea filtrelor de evenimente sau a metodelor de tratare a evenimentelor; ambele implementează interfața `javafx.event.EventHandler`.
- Acestea trebuie create și înregistrate pe nodurile corespunzătoare.
- Metodele de conveniență oferă un mecanism facil pentru tratarea evenimentelor. Acestea sunt definite în clasele `Node`, `Scene`, `Window` și subclasele acestora.
- MVC este un șablon de proiectare arhitectural utilizat pentru a separa responsabilitățile din cadrul aplicației.
- *Modelul* gestionează datele. *Viewul* prezintă datele utilizatorului. *Controllerul* mediază între model și view.

Rezumat II

- FXML este un limbaj declarativ bazat pe XML. FXML furnizează un mecanism pentru a separa *nivelul de prezentare* al aplicației de *nivelul logic* al aplicației. FXML face ușoară implementarea șablonului MVC.